Pilot Prototype Documentation

Introduction

The **Course Design Pipeline (CDP)** is a structured workflow system designed to manage collaborative course development between Tutors and the Content Creation Team.

The system separates:

- Subject expertise (Tutor)
- Instructional design and content production (Content Team)
- Final approval and governance (Tutor/Executive)

The goal of this pilot prototype is to demonstrate:

- End-to-end workflow integration
- Role-based responsibility separation
- Revenue governance logic
- Controlled lifecycle transitions
- Full frontend–backend–database integration

2. Objective of the Feature

The selected feature for implementation is:

Structured Course Design Lifecycle with Revenue Governance and Role-Based Visibility

This feature validates:

- Technical feasibility
- Usability of workflow
- Revenue logic automation
- Data access control

3. System Overview

The system follows a three-layer architecture:

Frontend (React + Tailwind)

↓

Backend API (Node.js + Express)

↓

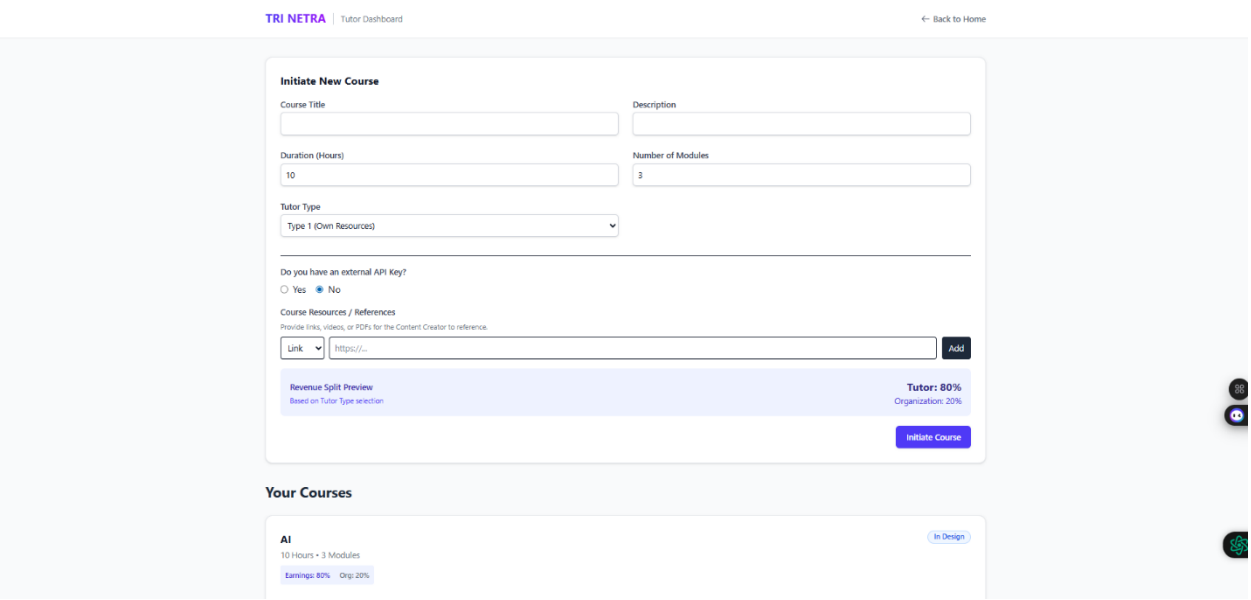
Database (PostgreSQL + Prisma ORM)

4. Roles and Responsibilities

Tutor

The Tutor is the subject expert responsible for:

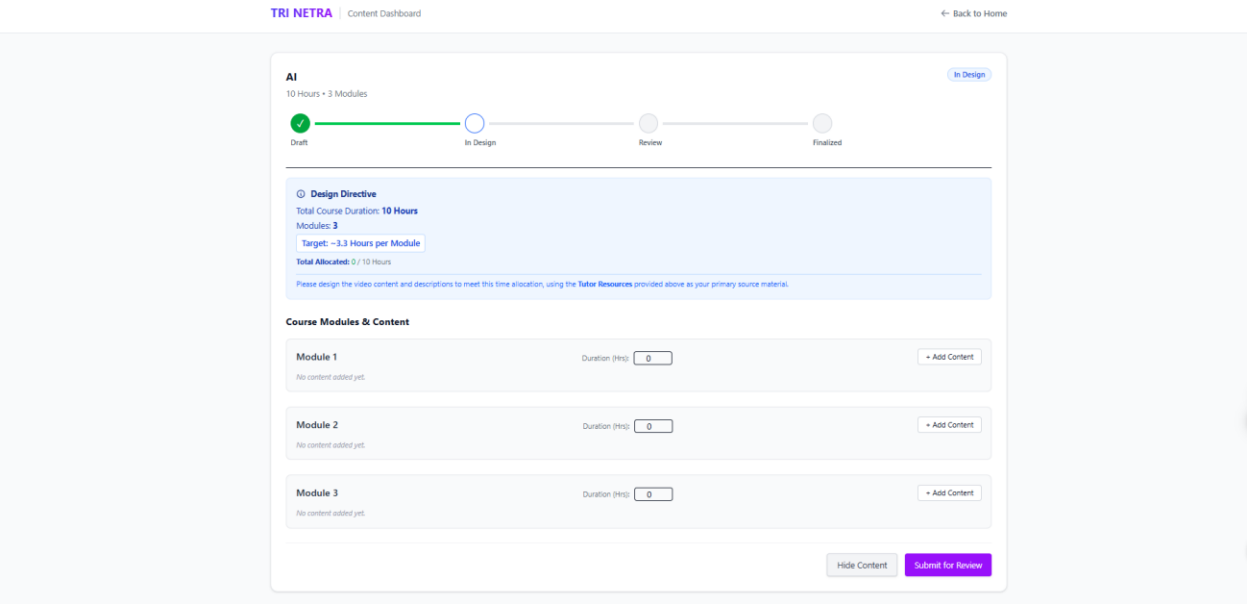
- Providing course title and description
- Defining number of modules
- Uploading resources (videos, materials, links)
- Selecting Tutor Type
- Reviewing final content
- Approving course for release
- Viewing revenue distribution



Content Creation Team

The Content Creation Team is responsible for:

- Structuring modules
- Developing content (if required)
- Uploading learning materials
- Moving course through workflow stages
- Submitting course for review



5. Tutor Types and Revenue Model

The system supports two tutor collaboration models.

Type 1 – Independent Tutor (Own API Key)

Characteristics:

- Tutor provides their own materials
- Tutor provides API key (optional AI usage)
- Content team structures the course

Revenue Distribution:

- Tutor → 80%
- Organization → 20%

Type 2 – Supported Tutor (No API Key)

Characteristics:

- Tutor provides description and requirements
- Content team creates full course content

Revenue Distribution:

- Tutor → 70%
- Organization → 30%

6. Workflow Lifecycle

The Course progresses through defined states.

CourseStatus Enum

- RAW_DRAFT
- IN_DESIGN
- EXECUTIVE_REVIEW
- FINALIZED

State Transition Rules

Current State	Next State	Role
RAW_DRAFT	IN_DESIGN	CONTENT
IN_DESIGN	EXECUTIVE_REVIEW	CONTENT
EXECUTIVE_REVIEW	FINALIZED	TUTOR

Invalid transitions are blocked by backend validation.

7. Database Design

User Table

- id
- name
- email

- role

Course Table

- id
- title
- description
- durationHours
- tutorType
- apiKey (nullable)
- revenueTutorPercent
- revenueOrgPercent
- status
- ownerId
- createdAt

Module Table

- id
- title
- order
- courseId

Resource Table

- id
- type (VIDEO, PDF, LINK)
- url
- courseId
- uploadedBy

DesignTask Table

- id
- type

- courseId
- assignedToId

8. API Design

POST /api/v1/pipeline/initiate

Role: TUTOR

Creates:

- Course
- Modules
- Revenue assignment

PATCH /api/v1/pipeline/:id/start-design

Role: CONTENT

Changes status → IN_DESIGN

PATCH /api/v1/pipeline/:id/submit-review

Role: CONTENT

Changes status → EXECUTIVE_REVIEW

PATCH /api/v1/pipeline/:id/approve

Role: TUTOR

Changes status → FINALIZED

GET /api/v1/pipeline

Role-based response filtering:

If role = TUTOR → return full course object

If role = CONTENT → remove revenue fields

9. Frontend Design

Routing Structure

- / → Landing Page
- /tutor → Tutor Dashboard
- /content → Content Dashboard

Tutor Dashboard Features

- Course creation form
- Tutor type selection
- Conditional API key input
- Resource upload (URL-based)
- Revenue split display
- Status badge display
- Approval controls

Content Dashboard Features

- View assigned courses
- View modules
- Upload content
- Move workflow stages
- Submit for review

Revenue information is not rendered here.

10. System Architecture

Client Layer

React + Tailwind UI

|

REST API

|

Application Layer

Express + Role Middleware

Business Logic + Prisma ORM

|

Database Layer

PostgreSQL

11. Sequence Flow

Course Creation

Tutor → POST /initiate → Backend → Database → RAW_DRAFT

Design Start

Content → PATCH /start-design → IN_DESIGN

Submit Review

Content → PATCH /submit-review → EXECUTIVE_REVIEW

Final Approval

Tutor → PATCH /approve → FINALIZED

---

13. Deployment Architecture

Development Setup

- React (Port 5173)
- Express (Port 5000)
- PostgreSQL (Port 5432)

14. Technical Challenges

- Prisma nested creation validation
- Role middleware configuration
- Revenue filtering logic
- State transition enforcement
- Module creation structure error

All issues were resolved through schema correction and controller validation.

15. Assumptions

- Header-based role validation (no JWT)
- Manual dashboard refresh

- Local development environment
- No payment gateway integration

16. Future Improvements

- Student purchase system
- Payment gateway integration
- Automated revenue tracking
- Authentication system
- Encrypted API key storage
- AI-assisted content generation
- Analytics dashboard

Course Design Pipeline Structure

Overview

The Course Design Pipeline facilitates collaboration between **Tutors** (Domain Experts) and **Content Creators** (Instructional Designers/Editors). It moves a course from an initial concept to a finalized product through a structured lifecycle.

Pipeline Lifecycle (State Machine)

Tutor InitiatesContent Creator StartsContent Creator SubmitsTutor ApprovesRAW_DRAFTIn Design QueueIN_DESIGNEXECUTIVE_REVIEWFINALIZED

detailed Stage Breakdown

1. RAW_DRAFT (Tutor)

Goal: Define the scope and requirements.

- **Actor**: Tutor
- **Actions**:
    - **Initiate Course**: Provide Title, Description, Total Duration (Hours), and initial Modules list.
    - **Set Tutor Type**:
        - TYPE_1 (Self-Provided Resources): Upload/Link resources (PDFs, Videos, Links).

        - TYPE_2 (Request Content): Define requirements for creators to research.
- **System State**: Course is visible to Content Creators but locked for editing until "Start Design".

2. IN_DESIGN (Content Creator)

Goal: Create specific content and allocate time.

- **Actor**: Content Creator
- **Pre-requisite**: Content Creator clicks "Start Design" (claims the task).
- **Actions**:
    - **Allocated Hours**: Input specific duration for each module to match the Course's Total Duration.
        - *Validation*: Total Module Hours should ideally match Course Duration.
    - **Add Content**: Upload videos or add links to specific modules.
- **Guidance**: "Design Directive" panel shows the target hours per module and total allocated time vs. budget.

3. EXECUTIVE_REVIEW (Tutor)

Goal: Quality assurance and final sign-off.

- **Actor**: Tutor
- **Actions**:
    - **Review Content**: Watch uploaded videos, check module durations.
    - **Approve**: Moves course to FINALIZED.
    - *(Future)*: Request Changes (Not yet implemented, currently manual feedback).

4. FINALIZED

Goal: Course is ready for students.

- **System State**: Read-only. Ready for publishing to Learning Management System (LMS).

Data Structure

Course Model

- id: UUID
- title: String

- durationHours: Float (Total Budget)
- status: Enum (RAW_DRAFT, IN_DESIGN, EXECUTIVE_REVIEW, FINALIZED)
- resources: List of references provided by Tutor.

Module Model

- id: UUID
- title: String
- duration: **Float (Allocated actual time)**
- contentItems: List of videos/materials.

Roles & Permissions

Action	Tutor	Content Creator
Initiate Course	✅	❌
View All Courses	✅	✅
Start Design	❌	✅
Edit Module Duration	❌	✅ (Only in IN_DESIGN)
Add Content Items	❌	✅ (Only in IN_DESIGN)
Submit for Review	❌	✅
Approve Course	✅	❌

Conclusion

The Course Design Pipeline pilot prototype successfully demonstrates:

- Structured collaboration workflow
- Contribution-based revenue model
- Role-based access control
- Controlled lifecycle management
- Full-stack integration

The system validates both technical feasibility and business scalability.